

# Accountable Anonymity for Public Money

Identity-Bound Bearer Payments: Mandatory Bilateral Non-Deniability Without a Trusted Opener

Perry Kundert

2026-06-11



Public money is going programmable, and programmability invites surveillance: every proposed central-bank digital currency couples the convenience of digital cash to a ledger its issuer can read, freeze, and mine. Physical cash resists this but offers the opposite extreme – no accountability, no issuance trail, no recourse. This paper presents BUCK Notes, a bearer-instrument layer for an identity-aware stablecoin that occupies neither pole. *Bulk* traceability is defeated – no analytics firm, no omnibus subpoena, not even the system’s own KYC issuer can reconstruct who paid whom – while *bilateral* disclosure is preserved: the two parties to any payment, and provably only they, retain the cryptographic means to prove under compulsion exactly who the counterparty was. The load-bearing security notion is the **non-deniable-receipt invariant**: there is no path, even for a *colluding* payer and payee, to construct an accepted payment for which no court-compellable receipt naming the KYC-bound counterparty exists. The invariant induces what we call a **compulsion calculus**: evidence of any payment always exists, is held exclusively and individually by its direct participants, and can be surrendered – or refused, or destroyed – only by them; no institution can be compelled to produce what it provably does not hold. The enabling primitive is **Identity-M binding**: payments name KYC-certified identity *points*, never account keys, realised by a single re-encryption gadget instantiated three ways across the full realisable design space of note flavours – all shipped and measured on Ethereum. We prove the fourth flavour structurally impossible, prove the obvious cheaper couplings vacuous, show that the doubly-anonymous flavour requires zero-knowledge registry set-membership under a stated restriction, and observe that the shipped construction lands on exactly the branch the necessity argument predicts. Every theorem carries an executable witness.

# Contents

|           |                                                                               |           |
|-----------|-------------------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Introduction</b>                                                           | <b>3</b>  |
| 1.1       | Contributions . . . . .                                                       | 4         |
| <b>2</b>  | <b>The Shape of the System, in Plain Language</b>                             | <b>5</b>  |
| <b>3</b>  | <b>Related Work</b>                                                           | <b>7</b>  |
| <b>4</b>  | <b>Model, Adversaries, and the Invariant</b>                                  | <b>8</b>  |
| 4.1       | Adversaries and privacy goals . . . . .                                       | 8         |
| 4.2       | The non-deniable-receipt invariant . . . . .                                  | 8         |
| 4.3       | Issuer-blindness: accountability without a bulk opener . . . . .              | 9         |
| 4.4       | The compulsion calculus . . . . .                                             | 9         |
| <b>5</b>  | <b>The Identity-M Principle</b>                                               | <b>11</b> |
| <b>6</b>  | <b>Construction</b>                                                           | <b>12</b> |
| 6.1       | The pool, the commitment, the nullifier . . . . .                             | 12        |
| 6.2       | Three flavours, one gadget . . . . .                                          | 12        |
| 6.3       | The spend-side composition . . . . .                                          | 13        |
| 6.4       | The A1 tie and the face-pinning principle . . . . .                           | 14        |
| 6.5       | The receipt, assembled . . . . .                                              | 14        |
| <b>7</b>  | <b>The Forced Taxonomy</b>                                                    | <b>15</b> |
| <b>8</b>  | <b>Security</b>                                                               | <b>16</b> |
| <b>9</b>  | <b>The Cost of Anonymity</b>                                                  | <b>18</b> |
| 9.1       | The EOA-transfer mirror: the cost <i>is</i> the anonymity . . . . .           | 18        |
| 9.2       | The obvious fix is vacuous . . . . .                                          | 18        |
| 9.3       | Necessity: set-membership, at one of exactly two moments . . . . .            | 18        |
| 9.4       | The shipped system is the predicted branch . . . . .                          | 19        |
| 9.5       | A mechanism remark: the recipient-burden posture . . . . .                    | 19        |
| <b>10</b> | <b>Implementation and Evaluation</b>                                          | <b>21</b> |
| 10.1      | What is shipped . . . . .                                                     | 21        |
| 10.2      | Costs (measured) . . . . .                                                    | 21        |
| 10.3      | Verification methodology and artifact . . . . .                               | 21        |
| <b>11</b> | <b>Discussion, Limitations, and Future Work</b>                               | <b>23</b> |
| <b>12</b> | <b>Appendix A: Executable Theorem Witnesses</b>                               | <b>24</b> |
| 12.1      | Setup: identities, accounts, registry . . . . .                               | 24        |
| 12.2      | Witness: Lemma 15 (key-equality coupling is vacuous) . . . . .                | 24        |
| 12.3      | Witness: Theorems 11 / 12 (A2 receipt; un-nameable is un-spendable) . . . . . | 25        |
| 12.4      | Witness: Theorem 10 (A2 layout – substitution cannot bind) . . . . .          | 25        |
| 12.5      | Witness: Theorem 10 (A1 layout – the public face pins the identity) . . . . . | 26        |
| 12.6      | Reproducing the full system . . . . .                                         | 26        |

# 1 Introduction

The choice a society makes about the privacy of its money is a choice about the balance of power between the individual and the institutions that watch over them.

That choice is now being made, mostly by default. Every central-bank digital currency under design or deployment couples the convenience of digital payment to a ledger that some institution can read in bulk – and the visibility is not an accident of engineering but the explicit pitch to the governments funding it<sup>1</sup>. Physical cash is the only payment instrument in wide use that resists bulk observation, and it resists by carrying *no* information at all: no issuance trail, no proof of payment a court could rely on, no recourse when it is stolen, and no per-instrument compliance hook. The public debate treats these as the only two points on offer – total legibility or total opacity – and forty years of cryptographic work has not yet displaced that false dichotomy from policy. Chaum asked the right question in 1982<sup>2</sup>; his answer stalled on institutional grounds, not cryptographic ones. The anonymity line that followed – Zerocash and its descendants, the mixer contracts, the compliance-retrofitted privacy pools<sup>3, 4, 5</sup> – delivered unlinkability and discarded accountability. The auditable-CBDC line that answered it – PRCash, UTT, PEReDi, Platypus<sup>6, 7, 8, 9</sup> – restored accountability, but in every case by vesting a *designated authority* (an auditor, a committee, the central bank itself) with the power to open transactions. No prior system provides both unlinkable payments and mandatory accountability with *no opener at all*.

This paper presents a system that does, and locates precisely what it costs. The design goal is civic before it is cryptographic, so we state it plainly:

- **Justice must work.** When a payment is implicated in a crime, the legal system must be able to obtain *cryptographically sound* evidence of exactly who paid whom – evidence strong enough to convict on and impossible to forge against an innocent party. The evidence must be guaranteed to *exist*: two colluding criminals must not be able to arrange an untraceable payment between themselves.
- **Compulsion must be targeted.** That evidence is held by – and may be demanded from – *only* the citizens directly party to the transactions at issue. There is no registry, exchange, issuer, or validator that a warrant (or a breach, or a purchase) can compel into revealing the financial actions of citizens in bulk, because no such party holds the information in any usable form.
- **Refusal must be possible.** If the legal order itself turns unjust – an authoritarian government criminalising beliefs, associations, donations – a citizen retains the physical ability to refuse: to withhold, or irreversibly destroy, their own keys. The data then *remains private*, against any coalition of compelled institutions and archived chain history, regardless of the

---

<sup>1</sup>Bank for International Settlements. "Rise of the Central Bank Digital Currencies." [TODO]

<sup>2</sup>Chaum, D. "Blind Signatures for Untraceable Payments." CRYPTO 1982. [TODO exact cite]

<sup>3</sup>Ben-Sasson, E., et al. "Zerocash: Decentralized Anonymous Payments from Bitcoin." IEEE S&P 2014. [TODO]

<sup>4</sup>Pertsev, A., Semenov, R., Storm, R. "Tornado Cash." 2019. [TODO]

<sup>5</sup>Buterin, V., Schaefer, J., Tromer, E., et al. "Blockchain Privacy and Regulatory Compliance: Towards a Practical Equilibrium (Privacy Pools)." 2023. [TODO]

<sup>6</sup>Wuest, K., Kostianen, K., Capkun, V., Capkun, S. "PRCash: Fast, Private and Regulated Transactions for Digital Currencies." FC 2019. [TODO verify authors/title]

<sup>7</sup>Tomescu, A., et al. "UTT: Decentralized Ecash with Accountable Privacy." IEEE S&P 2022. [TODO verify]

<sup>8</sup>Kiayias, A., Kohlweiss, M., Sarencheh, A. "PEReDi: Privacy-Enhanced, Regulated and Distributed Central Bank Digital Currencies." ACM CCS 2022. [TODO verify]

<sup>9</sup>Wuest, K., Kostianen, K., Delius, N., Capkun, S. "Platypus: A Central Bank Digital Currency with Unlinkable Transactions and Privacy-Preserving Regulation." ACM CCS 2022. [TODO verify]

preferences of the judiciary of the day. Surveillance of the unwilling is not available at any institutional price; it requires coercing individual human beings, one at a time, in the open.

These three goals are usually presumed to be in tension. The central claim of this paper is that they are simultaneously satisfiable, that a system satisfying them is practical today on a public blockchain, and that the property making it possible is a specific and somewhat counter-intuitive design inversion: payments must bind *identities* – KYC-certified curve points held by persons – and never the *account keys* that hold funds. We call this **Identity-M binding**. Everything in the paper flows from it: the receipt that always exists, the recovery of lost keys without any custodian, the impossibility of one whole quadrant of the design space, and the exact location of the boundary where accountability stops being cheap.

## 1.1 Contributions

- **The non-deniable-receipt invariant** (Def. 2): a security definition for payment systems whose operative, novel clause forbids a *colluding* payer/payee pair from constructing a payment with no accepting receipt. Accountability is **bilateral** and held by the counterparties; there is no system-wide opening authority.
- **The compulsion calculus** (Sec. 4.4): the invariant, composed with issuer-blindness, yields three properties we believe are jointly new for a payment system – evidence *exists* for every payment, is held *exclusively* by the direct participants, and is *refusable*: absent the participants' voluntary or compelled keys, the ledger is computationally silent forever. We argue this is the correct cryptographic substrate for due process, and the correct failure mode under unjust process.
- **Identity-M binding** (Sec. 5): the inversion that makes the rest practical. One gadget – "prove a re-encryption of a registered Identity under a target identity point" – instantiated three ways covers the full realisable design space; spend authorization keys on identity, so account-key loss is recoverable with no custodian, and the binding survives across the time gap that separates an anonymous payer from an anonymous payee.
- **A complete, shipped construction** for all three realisable flavours – addressed/public (A1), addressed/private (A2), bearer/public (B1) – on Ethereum: Groth16 batch mint, flavour-agnostic spend, spend-time composition of a deposit sigma, a bound set-membership proof, and a note-to-ciphertext tie, with every verifier real and measured (Sec. 10).
- **The forced taxonomy** (Thm. 8): the fourth flavour – a bearer note with a private issuer – is structurally impossible, by an information-theoretic argument.
- **A characterisation of the cost of anonymity** (Sec. 9): the receipt is nearly free when one party is named at the binding instant; when both are anonymous, the obvious cheap coupling is *vacuous* (Lem. 15) and a collusion-resistant receipt requires zero-knowledge registry set-membership under a single-binding-step restriction (Prop. 16). The shipped system takes exactly the spend-side branch the proposition permits, and we measure what it costs.
- **A reproducible, executable artifact**: the contracts, circuits, and verifiers are open and test-covered (over 400 Foundry tests, 330 wallet tests, an end-to-end suite in which every verifier is the real Groth16 artifact); the paper's own claims carry executable witnesses (Appendix 12) that run in this document's build.

## 2 The Shape of the System, in Plain Language

Before the formalism, the experience.

Alice owes Bob one hundred dollars. She opens her wallet and mints a *note*: her wallet burns one hundred of her BUCKs into a shared pool and publishes a single opaque number – a cryptographic commitment – onto the public chain. The commitment reveals nothing: not the amount, not Bob, not (for the private flavour) even that it was Alice. Alice then hands Bob the note's *opening* – a few hundred bytes – by any channel she likes: a message, a QR code on her phone's screen, a slip of paper. Days or months later, from any account he controls, Bob *deposits* the opening: his wallet proves, in zero knowledge, that some unspent note in the pool is his to claim, and the pool pays him one hundred BUCKs. An observer watching the chain sees a commitment appear at one moment and an unrelated-looking payout occur at another, with nothing connecting them – not the amounts (they're standard denominations shared by many Notes), not the parties, not the timing.

Three further facts make this different from a mixer, and they are the subject of this paper.

**First, the payment writes its own sealed receipt.** Built into every note – enforced by the chain's verifiers, impossible to omit – is the cryptographic material from which each party can produce a proof naming the *other party's certified legal identity*. Bob can prove "this payment came from Alice"; Alice can prove "Bob is the one who deposited it." The proof is sound: nobody can manufacture a receipt naming someone who was not actually party to the payment. And it is *unavoidable*: Theorem 12 shows that even if Alice and Bob conspire – both willing, both technically sophisticated – they cannot construct an accepted payment whose receipt material is missing or unusable. The worst a conspiracy can do is render a note addressed to nobody, and such a note is *unspendable*: the pool will not pay out on it.

**Second, only Alice and Bob hold the seal.** The receipt is not deposited with any registrar. It does not exist in any database. The KYC office that once certified Alice's identity cannot reconstruct her payments – we prove (Theorem 3) that even a *fully breached* identity office, holding every certificate it ever issued plus the entire blockchain, learns nothing about who transacted with whom. A court that needs the receipt has exactly one place to get it: from Alice, or from Bob, by lawful order directed at that person.

**Third, the seal answers to its holders – and only its holders.** Served with a lawful order, Bob can comply: he surrenders his receipt, and the court obtains evidence strong enough to convict on, evidence that mathematics – not testimony – ties to Alice. This is how the system serves justice: the victim of a fraud, the honest counterparty of a criminal, can always prove exactly whom they dealt with. But if the order is the instrument of an unjust regime – if the "crime" is a donation to a dissident, membership in a church, a purchase the state has decided to punish retroactively – Bob can refuse. He can decline to produce his key, accepting the personal legal consequence of that refusal; or he can have destroyed it long before, after verifying the payment, as a standing practice. Then *no one on earth* can open that payment: not the identity office, not the chain's operators, not a seizure of every server the system runs on. The mathematics does not know who is in power.

This is, we will argue, the correct shape for the money of a free society: per-payment accountability *stronger* than paper cash (which offers none), surveillance resistance *stronger* than bank money (which offers none), and a default the citizen controls with their own hands. Cash with a carbon-copy receipt, where the only copies belong to the people who made the payment.

The system comes in three flavours, fixed by who is named when:

- **A1 – addressed cheque, public payer.** The note names its recipient cryptographically (only Bob's identity can deposit it – a thief who photographs the paper gains nothing) and its issuer openly (payroll, invoices, audited disbursements).

- **A2 – addressed cheque, private payer.** As A1, but Alice’s identity is hidden from everyone *except Bob* – the flavour for paying privately while still handing the recipient a receipt that names you. This is the cryptographically expensive flavour, and Sections 9–10 are the story of what it costs and why.
- **B1 – bearer cheque, public payer.** Whoever holds the opening can deposit it (walking-around money; the paper surface), and the depositor’s identity is delivered – encrypted, bi-laterally – to the issuer. The fourth combination, a bearer note from a *hidden* issuer, is not merely undesirable; it is impossible (Theorem 8).

DRAFT

### 3 Related Work

We place BUCK Notes in the lineage of digital cash and privacy-preserving payment systems, and isolate the property no predecessor provides: a *mandatory, bilateral, collusion-resistant* receipt with *no system-wide opener*.

| System                                  | Unlinkable   | Mandatory receipt | Collusion-resistant | Trusted opener?          | Bearer/paper |
|-----------------------------------------|--------------|-------------------|---------------------|--------------------------|--------------|
| Chaum e-cash <sup>2</sup>               | issuer-blind | no                | no                  | central bank             | no           |
| Zerocash / Zcash <sup>3</sup>           | yes          | no (opt. viewkey) | no                  | no                       | no           |
| Tornado / Privacy Pools <sup>5</sup>    | yes          | no (innocence)    | no                  | no                       | no           |
| PRCash <sup>6</sup>                     | yes          | regulator-facing  | via auditor         | <b>yes</b> (auditor)     | no           |
| UTT <sup>7</sup>                        | yes          | budget-gated      | via committee       | <b>yes</b> (committee)   | no           |
| PEReDi <sup>8</sup>                     | yes          | regulator-facing  | via threshold       | <b>yes</b> (maintainers) | no           |
| Platypus <sup>9</sup>                   | yes          | regulator-facing  | via central bank    | <b>yes</b> (cen. bank)   | no           |
| CBDC (e-CNY, digital euro) <sup>1</sup> | no           | n/a               | n/a                 | issuer sees all          | no           |
| <b>BUCK Notes (A1, A2, B1)</b>          | <b>yes</b>   | <b>yes</b>        | <b>yes</b>          | <b>no</b>                | <b>yes</b>   |

The auditable-privacy line (PRCash, UTT, PEReDi, Platypus) is closest in spirit and the right comparison for a financial-cryptography audience: all four restore accountability to unlinkable payments, but all four vest a *designated authority* – an auditor, a committee, the central bank – with the power to de-anonymise, and that authority is exactly what a breached, purchased, or repurposed institution turns into a population-scale surveillance instrument. BUCK’s accountability is *counterparty-held*: the only institutional anchor is the per-identity KYC issuer, which (Theorem 3) can map a *surrendered* identity to a person but can never produce the link itself – compulsion runs depth-first through named individuals, never breadth-first across the system.

On the anonymity line, Zerocash-style view keys are *optional* and payer-controlled – precisely not mandatory – and the innocence-proving of Privacy Pools is voluntary exculpation, not compellable evidence. On the bearer axis, prior physical-form digital cash (Casascius coins, BIP-38 paper wallets) wraps a bearer secret with no identity semantics at all; BUCK’s addressed flavours make the paper artifact itself useless to anyone but the named recipient.

## 4 Model, Adversaries, and the Invariant

An *Identity* is a KYC-certified point  $M = mG \in \mathbb{G}_1$  (BN254), where the scalar  $m = H(\text{id})$  is derived by a preimage-resistant hash from the certified attribute record  $\text{id}$ . A person registers any number of on-chain *accounts*; each account  $X$  stores its identity ElGamal-encrypted under its own key,  $(E_{\{\text{addr}\}}[X] = (R_X, C_X) = (r_X G, M_X$

- $r_X \setminus, pk_X) \setminus$  with  $pk_X = sk_X G$ , accompanied by a

rerandomised Pointcheval-Sanders credential proving the identity was certified – without revealing which certificate. The registry additionally maintains an incremental Merkle accumulator over identity points (root  $rt_{\text{id}}$ , Poseidon leaves), updated commit-before-use as accounts register. A *payment* moves value  $v$  from payer  $P$  to recipient  $D$ .

### 4.1 Adversaries and privacy goals

**Definition 1.** *Three properties, held against adversaries of increasing power:*

- (**Bulk-tracing resistance.**) Mallory, a passive observer with full chain history and unbounded compute but no party’s secrets, cannot reconstruct who paid whom, how much, or link a mint to its spend(s).
- (**Third-party non-disclosure.**) Nothing on chain reveals either party’s Identity point  $M$  to a non-participant, even in aggregate across many payments – including to the issuer (below).
- (**Mutual decryptability / accountability.**) Each of the two participants, from data they hold plus public chain state, can produce a sound receipt naming the other’s registered  $M$ ; and a colluding pair cannot arrange an accepted payment for which no such receipt exists.

The first two properties must hold against the strongest realistic deanonymiser, the **Issuer**: the KYC trust anchor that holds, for every client it certified, the plaintext identity  $(\text{id}_i, m_i, M_i)$  and the credential  $\sigma_i$  it issued – strictly more than Mallory knows. We treat a compelled or wholly breached issuer in Sec. 4.3.

### 4.2 The non-deniable-receipt invariant

**Definition 2.** (Non-deniable receipt.) *A protocol satisfies the **non-deniable-receipt invariant** if for every accepted payment there exist a receipt  $\mathcal{R}$  and an efficient public predicate  $\text{RcptVerify}$  such that:*

- (**Completeness.**) *The recipient  $D$  can compute  $\mathcal{R}$  from data they hold plus public chain state, and  $\text{RcptVerify}(\mathcal{R}) = 1$  names a registered Identity  $M_P$  and value  $v$ .*
- (**Soundness / non-frameability.**) *No PPT adversary produces  $\mathcal{R}'$  with  $\text{RcptVerify}(\mathcal{R}') = 1$  naming an Identity that did not author the payment.*
- (**No deniability.**) *No PPT payer can cause an accepted payment to  $D$  for which no accepting receipt naming the payer exists – even in collusion with  $D$ .*

The third clause is the operative one and the departure from prior art: it is the collusion-resistant form. Completeness alone protects a self-interested recipient; the invariant additionally forbids a *willing* pair from constructing an untraceable payment. Symmetric statements bind the recipient’s identity toward the payer (the exact per-flavour content is Theorem 11).

### 4.3 Issuer-blindness: accountability without a bulk opener

The KYC issuer is the system’s only trust anchor, and the natural objection to any *accountable privacy* claim is that the issuer is a bulk-deanonymisation backdoor. It is not. Issuance is *address-blind*: the issuer signs the identity scalar  $m = H(\text{id})$  once (a Pointcheval-Sanders credential over  $m$  alone) and is thereafter done; the client derives each on-chain account *offline* – a fresh key pair  $(sk, pk)$ , a self-formed ElGamal credential  $E_{\text{addr}} = (rG, M + rpk)$ , and a *rerandomised* credential  $\sigma'$  – without the issuer ever seeing  $pk$ . The issuer’s view is therefore, per certified client  $i$ , the tuple  $(\text{id}_i, m_i, M_i, \sigma_i)$  – the plaintext identity – plus public chain state; it holds *no* chain addresses.

**Theorem 3.** (Issuer-blindness.) *Under statistical unlinkability of PS rerandomisation and IND-CPA security of ElGamal on  $\mathbb{G}_1$  (equivalently DDH), an issuer – even one compelled or wholly breached, in possession of  $\{(\text{id}_i, m_i, M_i, \sigma_i)\}_i$  and the entire chain – has negligible advantage in matching any issued identity  $M_i$  to its on-chain registration  $(pk_j, E_j, \sigma'_j)$ . Compulsion or breach yields the set of certified identities but no map to chain accounts; deanonymisation remains per-account, requiring that account’s  $sk$  or a counterparty receipt.*

*Proof.* Two locks. *Credential lock.*  $\sigma'_j$  is a rerandomisation of some issued  $\sigma_i$ ; by statistical unlinkability the two carry zero mutual information, so even an unbounded issuer cannot recognise its own signature. *Plaintext lock.* The issuer *knows*  $M_i$ , so its task is not decryption but *plaintext-checking*: does  $E_j = (R, C)$  encrypt  $M_i$ ? That is deciding whether  $(G, pk_j, R, C - M_i)$  is a DDH tuple, and an algorithm answering it for a chosen  $M_i$  breaks IND-CPA (submit  $m_0 = M_i, m_1$  random). Knowing the plaintext thus confers no advantage. With both locks shut and no stored  $pk$ , the issuer is reduced to an independent DDH decision per (identity, account) pair.  $\square$

This is the load-bearing form of *no trusted opener*: the closest thing to an authority cannot open transactions in bulk, even against itself – under compulsion it can only *verify* a link some party surrenders (map a receipt-disclosed  $M$  to the real person), never *produce* one. Two caveats bound the theorem, and we state rather than bury them:

- **Computational, not information-theoretic.** The plaintext lock is DDH; only the credential lock is statistical. A discrete-log break – in particular a quantum computer running Shor against  $\mathbb{G}_1$  – turns the harvested  $\{M_i\}$  plus the archived chain into a wholesale, retroactive deanonymisation of *past* identity-bound traffic. The breached or compelled issuer’s KYC records are precisely that harvest corpus. Migration of the identity substrate ahead of crypt-analytic risk is a deployment obligation, not an afterthought (Sec. 11).
- **Retroactive, not prospective.** Address-blind issuance protects *past* registrations unconditionally against *future* issuer compromise; but a coerced issuer can change the protocol going forward (demand  $pk$  at KYC, record addresses, backdoor credentials). Issuer-blindness is a statement about issuances already performed under the honest protocol.

### 4.4 The compulsion calculus

The invariant and issuer-blindness, composed, induce a precise answer to the question *who can a court reach, and what happens when they refuse?* We state it as three properties. Call a party’s *opening material* for a payment the secrets from which an accepting receipt can be computed: the note opening (its randomness  $\rho$  and identity payload) together with, per flavour, the holder’s identity scalar or account key needed to decrypt the counterparty ciphertext.

**Proposition 4.** (Existence and exclusivity.) *For every accepted payment: (i) accepting receipts naming each counterparty exist and are computable by the other counterparty from material they hold (Def. 2, clauses 1 and 3; Theorem 11); and (ii) no PPT party outside the payment’s direct participants – including a coalition of Mallory and the compelled issuer of Theorem 3 – can compute any accepting receipt for it, under the privacy properties of Theorem 14.*

**Proposition 5.** (No institutional bypass.) *The only compellable institution, the KYC issuer, can perform exactly one useful act under compulsion: given an identity point  $M$  already disclosed by a participant’s receipt, it maps  $M$  to the certified legal person. It cannot – even fully breached – enumerate, sample, or statistically approximate the payments of any person (Theorem 3). No other system component (contracts, verifiers, provers, relayers) holds any secret at all.*

**Proposition 6.** (Refusal-resilience.) *If every direct participant in a payment withholds or destroys their opening material, then no PPT coalition of all remaining actors – the compelled issuer with its full KYC corpus, all chain history, all registry state – gains non-negligible advantage in identifying either party or linking the payment’s mint to its spend. Destruction is irreversible: the opening material is the only trapdoor, and it never existed anywhere but in the participants’ custody.*

Propositions 4–6 follow from the privacy theorems (Sec. 8) plus issuer-blindness; their content is the *composition*, which is worth reading as constitutional design rather than cryptography:

- **Due process is fully served.** Evidence exists for every payment (no colluding pair can prevent it), it convicts only the guilty (non-frameability: a receipt cannot be forged against an innocent party – Corollary 13), and a court always has a *specific person* to direct an order at. Investigation proceeds the way warrants are supposed to work: from a known participant, one named counterparty at a time, each step producing sound evidence.
- **Bulk surveillance is not for sale.** There is no database to subpoena, breach, or buy. The marginal cost of surveilling one more citizen is coercing one more citizen – publicly, individually, through legal process directed at a person who can see it happening. The system makes population-scale financial monitoring not illegal but *unimplementable*.
- **The last word belongs to the citizen.** Refusal has personal legal cost (contempt, adverse inference) – as it should, under legitimate process. But it is *effective*: a citizen facing an unjust order, or a population facing a judiciary that has begun criminalising beliefs, can withhold or destroy keys, and the archived ledger stays dark regardless of what any institution is later compelled to do. The privacy floor survives institutional capture because no institution was ever load-bearing.

Two honest edges. First, receipts are bilateral: your counterparty’s copy is beyond your control – *by design*. A criminal cannot silence their victim’s receipt by destroying their own; refusal protects a payment only when every participant elects it, which is exactly the case (consensual acts the regime of the day has criminalised) the property is for. Second, destroying your keys also destroys *your* evidence: the standing ability to prove what you paid and to whom. Refusal-resilience is an option with a price, chosen by its holder – not a default that degrades accountability for anyone else (Def. 2 holds for every accepted payment regardless).

## 5 The Identity-M Principle

The properties above are only as good as the binding that produces the receipt, and the central design decision is *what* a payment binds. The obvious answer – the counterparty’s account public key, the thing that actually signs transactions – is wrong, and the system’s history includes the scar: an early account-pinned design bound A-flavour notes to the recipient’s mint-time key pair via an in-circuit Chaum-Pedersen equality, with the consequence that losing that one key made the note permanently unspendable, and any key rotation or multi-device custody broke addressing. Worse, Sec. 9 shows the cheap forms of key binding are not merely brittle but *vacuous*: an ElGamal ciphertext does not commit to its key, so "encrypted under the same key" constraints are absorbed by free witnesses (Lem. 15).

The correct authority axis is the **identity point**  $M = H(\text{id})G$ :

- **Identities are scarce and non-grindable; keys are free.** Anyone can mint key pairs at will, so key possession proves nothing about *who*. A registered  $M$  exists only by KYC certification, and its preimage structure denies an adversary any choice of its value:

**Definition 7.** (Non-grindable identities.) *Each registered identity point is  $M = H(\text{id}) \cdot G$ , where  $\text{id}$  is the KYC-verified attribute record and  $H$  (keccak256 over a canonical encoding) is preimage-resistant, modelled as a random oracle. No PPT adversary registers an identity point of its choosing; in particular it cannot register an account whose  $M$  equals a prescribed target  $M' + \delta G$  for an adversarially chosen  $\delta \neq 0$ .*

- **Accounts are envelopes; the identity is the addressee.** A note addressed to  $M_{\text{rec}}$  is spendable from *any* account whose registration binds it to  $m_{\text{rec}}$  – including one created after the note was minted. Key loss is recovered by registering a fresh account to the same identity; no custodian, no social recovery, no issuer involvement. Dually, the receipt names the *person* (their certified  $M$ ), not a disposable key.
- **Identity binding survives the time gap.** The deep obstruction in doubly-anonymous payments (Sec. 9) is that the payer’s anchor exists only at mint and the recipient’s only at spend; no single on-chain moment sees both. Account keys cannot bridge the gap (the recipient’s spending key may not exist yet at mint). The identity point can: it is stable across the gap, and – the technical heart of the system – *everything a note must say about its parties can be said as an ElGamal re-encryption statement about identity points*.

The last observation collapses the construction into **one gadget**:

**The gadget.** Prove, in zero knowledge, that a ciphertext is an encryption (or re-encryption) of a *registered* Identity under a target identity point – where "registered" is certified by Merkle membership of the underlying point in the registry accumulator  $\text{rt}_{\text{id}}$ , bound to the same blinded commitment  $P = M + bH$  the accompanying sigma protocol opens.

Three instantiations of the gadget – differing only in *which* identity is encrypted under *whose* point, and at which moment – produce the three realisable flavours (Table in Sec. 6). The same inversion gives the spend path its shape: at deposit, the depositor proves *their account is bound to the identity the note addresses* – a statement about  $m$ , checked by a cheap sigma protocol against the account’s registered ciphertext, composed with a set-membership proof that the named point is real, and a SNARK tie that the whole gate is about *this* note. Authorization keys on the identity; the account merely carries it.

## 6 Construction

A standard transfer between two registered identities is a synchronous bilateral handshake: each party re-encrypts their credential under the other’s key and proves (Chaum-Pedersen) the re-encryption carries the same  $M$ . Both identity anchors are co-present in one call; mutual decryptability is free. A Note reorganises the same handshake through a commitment pool, splitting it across time so that mint and spend are unlinkable – and the rest of the construction is the work of carrying the bilateral binding across that split.

### 6.1 The pool, the commitment, the nullifier

The pool is a Tornado-style Poseidon Merkle tree of note commitments with a rolling root window and a nullifier set. A note is the opening tuple; its on-chain commitment is

$$cm = \text{Poseidon}_5(\text{flavor}, v, \rho, idHash, \text{predicate}),$$

where  $\rho$  is the note’s secret randomness and  $idHash$  is a Poseidon hash of the flavour’s *identity payload* – the commitment to everything the note says about its parties. Batch mints are proven by a Groth16 circuit (per pinned batch size  $N$ ) that opens every commitment, range-checks values, sums them to the public escrowed total, and folds the leaves into the SNARK-attested new root; the contract escrows exactly the proven total in the same atomic call.

Spends are proven by a single flavour-agnostic circuit: the prover shows knowledge of an opening of *some* tree leaf under a recent root, with public (root,  $nf$ , face, recipient, chainid), where the nullifier

$$nf = \text{Poseidon}_3(\rho, idHash, 4242)$$

is one derivation for every flavour (the constant domain-separates nullifier preimages from commitment preimages). Double-spends are rejected by set membership. No account key enters the hash: spend *authorization* is layered per flavour on top of the nullifier, which is what makes key-loss recovery and account-agnostic deposit possible (Sec. 5) – and what creates the binding obligation the remainder of this section discharges.

### 6.2 Three flavours, one gadget

What  $idHash$  commits, and which direction of the gadget runs at which moment, defines the flavour:

| Flavour | idHash commits                       | Issuer named...                              | Recipient named...                                 |
|---------|--------------------------------------|----------------------------------------------|----------------------------------------------------|
| A1      | $(eNote, m_{iss}, \sigma_{Schnorr})$ | at mint, in the clear (batch Schnorr)        | at spend: coupling to $M_{rec}$ via $eRec$         |
| A2      | $(eNote, eIss)$                      | at spend, to recipient alone: decrypt $eIss$ | at spend: same coupling, target $M_{rec}$          |
| B1      | $(m_{iss}, \sigma_{Schnorr})$        | at mint, in the clear                        | at spend: depositor binding $eDepForIss$ to issuer |

with the per-flavour ciphertexts (all ElGamal over identity points):

- **A2 (addressed, private issuer).** The issuer encrypts *its own* registered identity under the recipient’s identity point:  $eIss = (r'G, M_I + r'M_{rec})$ , decryptable by  $m_{rec}$  alone; the note value rides in  $eNote = (r_nG, vG + r_nM_{rec})$ . At mint, a blinded re-encryption sigma (`verifyIssuerReenc`) proves  $eIss$  re-encrypts the *minting account’s registered* identity – the anti-framing anchor – without revealing it; each leaf’s  $eIss$  is also a public input of the A2 mint SNARK (the *leaf-tie*), so no leaf can lack a binding and no binding can float to another leaf.

- **A1 (addressed, public issuer).** The issuer is named openly (a Schnorr signature over the batch under their registered key, checked at mint), and the recipient's identity is encrypted *under itself*:  $eRec = (r'G, M_{rec} + r'M_{rec})$ , so exactly the addressed identity can open and answer for it.
- **B1 (bearer, public issuer).** No recipient exists at mint (Theorem 8 makes this the *only* coherent bearer cell); at spend the depositor encrypts their own identity under the public issuer's key,  $eDepForIss = (rG, M_{dep} + rpk_{iss})$ , and proves it carries the same identity their payout account is bound to.

### 6.3 The spend-side composition

Every addressed deposit (`spendCoupledA1`, `spendCoupledA2`) verifies, in one transaction, the flavour-agnostic note proof plus three mutually-bound gates:

1. **The deposit-coupling sigma** (`verifyDepositCoupling`, EIP-196 arithmetic): the depositing account `msg.sender` is registered and bound to a scalar  $m$ , and the supplied ciphertext  $eEnc$  decrypts under  $m$  to the point blinded inside the commitment  $P_I = M + bH$  – a perfectly-hiding Pedersen-style point published by the sigma.
2. **The bound membership proof** (`identity_membership_g1tie`, Groth16):  $P_I$ 's underlying point is a member of the registry accumulator `rtid`. *Bound* is load-bearing: the verifier derives its public inputs from the same  $P_I$  the sigma committed, so the two gates cannot be answered with different points.
3. **The note  $\leftrightarrow$  eEnc tie** (`note_binding` / `note_binding_a1`, Groth16): the ciphertext the first two gates verified is keyed to the identity material committed in the `idHash` behind *this spend's nullifier* – the gate is about this note, not a depositor-substituted ciphertext. The tie shares the nullifier with the spend proof and  $P_I$  with the sigma.

```
// inside Notes._spendCoupled (abridged; A2 passes a1Layout=false, A1 true):
require(spendVerifier.verifySpend(proof, root, nullifier, face, recipient, chainid));
require(reg.verifyDepositCoupling(msg.sender, eEnc, dc), "bad deposit coupling");
//                                     dc.P_I --- the same point ---v
_verifyIdentityMembership(membershipProof, dc.P_I.X, dc.P_I.Y);
_verifyNoteBinding(noteBindingProof, nullifier, a1Layout, face,
                  eEnc, dc.P_I.X, dc.P_I.Y);
```

The composition discipline – every proof's public inputs derived on chain from the same objects (`msg.sender`,  $eEnc$ ,  $P_I$ , the nullifier, the face), never taken from the prover's bytes – is what makes the gates impossible to decouple under collusion, and it is the pattern we commend to designers beyond this system. For B1 the depositor-binding sigma (an Okamoto-style proof coupling the payout account,  $eDepForIss$ , and  $P_{dep} = M_{dep} + bH$ ) replaces gate 1, the same bound membership is gate 2, and no tie is needed (the depositor names *itself*; there is nothing to substitute).

For A2,  $eEnc$  is a fresh *re-encryption* of the committed  $eIss$  (same plaintext, new randomness): re-using the mint-time ciphertext – a public input of `Notes.mint` – would link spend to mint and destroy the flavour's anonymity. Hence the tie proves re-encryption *equivalence*, never equality. For A1, the committed payload has no second ciphertext, so the tie runs through the note's own value ciphertext with the face public; the next subsection explains why that public face is not incidental but the soundness linchpin.

## 6.4 The A1 tie and the face-pinning principle

ElGamal ciphertexts are not key-committing: given the randomness  $r_n$  (which travels with the opening), the "key" of  $eNote = (r_n G, C_n)$  can be re-explained as any  $m'$  by absorbing the difference into a free plaintext,  $V' = C_n - (r_n m')G$ . A binding circuit that witnesses the plaintext freely therefore proves nothing about the addressed identity. The A1 tie pins the plaintext *publicly*: its statement fixes  $eNote.C = (v + r_n m_{\text{rec}})G$  with  $v$  a public input that the contract sets to the spend's **face** – itself bound to the note's committed value by the spend SNARK over the same nullifier. With  $v$  and  $r_n = \log_G eNote.R$  determined,

$$m_{\text{rec}} = (\log_G eNote.C - v) \cdot r_n^{-1}$$

is *unique*: only the identity the note was addressed to can satisfy the relation (Theorem 10, A1 clause; executable witness in Appendix 12). The observation generalises: when a binding must travel through a non-committing encryption, *publishing the plaintext's already-public component is the cheapest commitment*.

## 6.5 The receipt, assembled

A receipt is a four-link proof chain, every link checkable by a third party from public state plus the holder's disclosed material: (a) *opening* – the disclosed tuple re-hashes to a commitment; (b) *minted* – that commitment is a leaf under a historical root; (c) *issuer* – the flavour's issuer binding (Schnorr in the clear, or the decrypted-and-membership-certified *eIss*); (d) *paid* – the nullifier consumed and face transferred. The recipient-naming direction is the spend-side composition above. Link (c) is where the invariant historically lives or dies, and the whole of Sec. 9 is the anatomy of keeping it alive when the issuer is anonymous.

## 7 The Forced Taxonomy

Who can be named, and how, is fixed by what is known at mint time – the axis that defines the flavours. One of the four nominal cells (bearer  $\times$  private issuer, "B2") is not merely undesirable but impossible.

**Theorem 8.** (No private-issuer bearer note.) *No bearer note can both (i) deliver its issuer's identity to its eventual redeemer (mutual decryptability) and (ii) withhold its issuer's identity from non-redeeming holders and observers (issuer privacy).*

*Proof.* The argument is information-theoretic. A bearer note's redeemer is unknown at mint: it is whoever comes to hold the opening, which circulates among holders by construction. For the redeemer to name the issuer, the note must carry an issuer-identity binding that the redeemer can open; the issuer creates that binding at mint, when the redeemer is unknown, so it must be openable under key material the eventual redeemer will hold. Exactly two sources of such material exist. (1) *The redeemer's own registered secret:* but the issuer cannot target a key or identity point it has never seen, and "whichever member of the public eventually holds the paper" names no point – targeting one would require predicting the redeemer, i.e. the note would be addressed, not bearer. (2) *Note-bound material derived from the opening itself* (e.g. a key derived from  $\rho$ ): then *every* holder along the chain of custody can open the binding, so the issuer's identity is disclosed to all holders – and a bearer instrument's holders are, in the limit, the public – violating (ii). A binding openable by neither is openable by the redeemer in particular, violating (i). Hence a hidden issuer requires a designated recipient: bearer and private-issuer are mutually exclusive.  $\square$

The contrast with A2 is exact: A2 reveals the issuer to *one chosen party* by encrypting under a point only that party's identity scalar opens – which is possible precisely because the recipient is designated. The realisable set is therefore  $\{A1, A2, B1\}$ , and the implementation enforces the boundary structurally: the mint circuit's per-leaf issuer-mode signal admits no private-issuer bearer codepoint, so a B2 leaf is not merely rejected but *unprovable*.

## 8 Security

We state the results at the level a reader needs to audit the claims of Secs. 4–6; the full reductions are Theorems 1–12 of the companion *Proofs* document, and each result here carries an executable witness (Appendix 12). Assumptions: discrete log / DDH on BN254  $\mathbb{G}_1$ ; Poseidon collision-resistance and PRF behaviour (ROM); Groth16 knowledge-soundness and zero-knowledge; Schnorr/Okamoto special soundness; an independent NUMS point  $H$  (no known  $\log_G H$ ); and Assumption 7.

**Theorem 9.** (Deposit gate soundness.) *If a coupled spend is accepted, then except with negligible probability there exist extractable  $(m, sk_{\text{dep}}, b)$  such that: the depositing account is registered with credential binding it to identity scalar  $m$ ; the supplied  $e\text{Enc}$  decrypts under  $m$  to the point  $M^*$  committed in  $P = M^* + bH$ ; and  $M^*$  is a member of the registry accumulator  $\text{rt}_{\text{id}}$ . The same  $P$  satisfies all three clauses: answering the sigma with one point and the membership with another requires  $\log_G H$ .*

**Theorem 10.** (Note  $\leftrightarrow$   $e\text{Enc}$  tie.) *If the binding proof is accepted for nullifier  $nf$ , then except with negligible probability the extractor produces an opening  $(\rho, \text{idHash}, \dots)$  with  $nf = \text{Poseidon}_3(\rho, \text{idHash}, 4242)$  – the very nullifier the spend consumed – and:*

- **(A2 layout.)**  $\text{idHash} = \text{Poseidon}_8(e\text{Note}, e\text{Iss}_0)$ , the supplied  $e\text{Enc}$  is a re-randomisation of the committed  $e\text{Iss}_0$  under  $M_{\text{rec}} = m_{\text{rec}}G$ , the committed ciphertext decrypts under  $m_{\text{rec}}$  to  $M_I$ , and  $P_I = M_I + bH$  is the sigma’s committed point.
- **(A1 layout.)**  $\text{idHash} = \text{Poseidon}_8(e\text{Note}, m_{\text{iss}}, \sigma)$ ,  $e\text{Note} = (r_nG, vG + r_nM_{\text{rec}})$  with  $v$  the spend’s public face,  $e\text{Enc}$  is a fresh encryption of  $M_{\text{rec}}$  under itself, and  $P_I = M_{\text{rec}} + bH$ . The public  $v$  makes  $m_{\text{rec}}$  unique (Sec. 6); a holder of a stolen opening with identity  $m' \neq m_{\text{rec}}$  has no satisfying witness.

Composed with Theorem 9 over the shared  $P_I$ , the depositor’s authenticated identity is the identity the spent note addressed, and (A2) the membership-certified point is the committed ciphertext’s decryption.

**Theorem 11.** (Mutual decryptability, all flavours.) *For every accepted payment, each party can produce an accepting receipt naming the other’s registered identity: A1/B1 recipients name the issuer from the in-clear  $M_{\text{iss}}$  plus batch Schnorr; A2 recipients decrypt  $e\text{Iss}$  under  $m_{\text{rec}}$ , with the mint binding and membership certifying it is the registered minter; A1/A2 issuers hold the addressed  $M_{\text{rec}}$  from mint; B1 issuers decrypt  $e\text{DepForIss}$ . All receipts are third-party checkable.*

**Theorem 12.** (No deniability under collusion.) *No PPT payer/recipient coalition produces an accepted payment with no accepting receipt. At mint, every leaf must carry a verified issuer binding (Schnorr or blinded re-encryption + leaf-tie; the circuit’s issuer-mode signal leaves no unbound codepoint). At spend, the deposit gate (Thms. 9, 10) reverts unless the verified ciphertext – provably the note’s own – decrypts under the depositor’s authenticated identity to a registry member. A coalition can mint a note whose committed identity material is garbage, but such a note admits no membership witness and is unspendable: un-nameable implies un-spendable.*

**Corollary 13.** (Deny, never frame.) *The only deviation available to a coalition is to render a note un-nameable – in which case it cannot be spent – never to make a receipt name an innocent third party: every binding is proven over  $\text{msg.sender}$ ’s own registered credential (mint) or the depositor’s own account binding (spend), and Assumption 7 denies the coalition any registered identity at an offset of its choosing. Receipts are therefore evidence-grade: an accepting receipt names a party that actually acted.*

**Theorem 14.** (Privacy.) *Against Mallory and against the (even breached) issuer: commitments and nullifiers reveal nothing (Poseidon hiding/PRF); a spend links to its mint only through  $nf$ , whose preimage includes the secret  $\rho$ ; the spend-side sigmas are HVZK and their only identity-derived public value is the perfectly-hiding  $P = M + bH$ ; the membership and tie proofs are Groth16 zero-knowledge with public inputs derived from already-public values;  $eEnc$  is a uniform re-randomisation. The residual public surface of a spend is exactly (face, recipient account) – mitigated operationally by fixed denominations and relayed payouts – and of a mint, the batch total.*

Proof sketches: Theorem 9 is Okamoto special soundness for the sigma plus Groth16 knowledge-soundness for the membership, glued by the on-chain derivation of both proofs' public inputs from one  $P$  (any discrepancy yields  $\log_G H$ ). Theorem 10 is Groth16 knowledge-soundness over the two binding relations; the A1 uniqueness clause is the face-pinning argument. Theorems 11–14 and the corollary compose these with the mint bindings and the identity-layer results (PS unlinkability, ElGamal IND-CPA, Theorem 3). Full statements: *Proofs* Theorems 7–12.

DRAFT

## 9 The Cost of Anonymity

The public-issuer flavours bind the issuer with a signature and the depositor with sigma protocols – cheap EIP-196 arithmetic, no SNARK over identities at all beyond the shared membership proof. What makes the doubly-anonymous flavour (A2: issuer named only at mint, recipient only at spend, the two unlinkable) categorically harder? This section locates the boundary exactly: first the structural diagnosis, then the vacuity of the obvious fix, then the necessity result, and finally the observation that the shipped architecture is the necessity result’s permitted branch, made practical.

### 9.1 The EOA-transfer mirror: the cost *is* the anonymity

A direct transfer between registered accounts achieves mutual decryptability with free collusion-resistance because the mandatory Chaum-Pedersen-bound approve reads *both* parties’ registered records in a single call: both anchors co-present, both named. A2 splits the anchors in time *by demand of its privacy goal*: the issuer’s anchor is live only at mint, the recipient’s only at spend, and unlinkability forbids any public value from bridging them. The A2 mint binding is, line for line, the approve handshake with one change: the recipient’s identity is *blinded*. That one change is the entire gap – it surrenders the phrase “under the recipient’s *registered* key” – and everything that follows is the price of buying the phrase back without un-blinding anything.

### 9.2 The obvious fix is vacuous

**Lemma 15.** (Vacuity of key-equality coupling.) *Let an A2 mint constrain only that the issuer payload  $eIss$  and the note ciphertext  $E_{note}$  are encrypted under the same key  $pk_{rec}$ , with the recipient’s identity point  $M_{rec}$  a free witness. Then for any issuer-chosen  $pk_{bogus}$  the constraint is satisfiable while the note remains spendable by the intended recipient  $D$  and  $eIss$  is encrypted under  $pk_{bogus} \neq pk_D$ .*

*Proof.* Set  $pk_{rec} := pk_{bogus}$  and  $M_{rec} := M_D + r_n(pk_D - pk_{bogus})$ . Then  $E_{note} = (r_nG, M_{rec} + r_npk_{rec}) = (r_nG, M_D + r_npk_D)$ , a valid encryption of  $M_D$  under  $D$ ’s registered key, so the spend-side check accepts and  $D$  spends; while  $eIss = (r'G, M_{iss} + r'pk_{bogus})$  is under  $pk_{bogus}$ , so a compelled  $sk_D$  decrypts  $eIss$  to  $M_{iss} + r'(pk_{bogus} - pk_D) \neq M_{iss}$ . The free witness  $M_{rec}$  absorbs the constraint. (Executable witness: Appendix 12.)  $\square$

The lemma is the formal autopsy of an entire class of cheap designs – and the second motivation (after key loss) for the Identity-M inversion: binding to a key is worthless because keys are free and ElGamal does not commit to them. The binding must reach a *registered* identity, and the issuer – who holds no recipient secret – can assert registration only as a set-membership statement.

### 9.3 Necessity: set-membership, at one of exactly two moments

**Proposition 16.** (Conditional necessity.) *Let  $\Pi$  be a protocol that, in the A2 setting, (i) preserves bulk-tracing resistance and third-party non-disclosure (issuer named only at mint, recipient only at spend, mint and spend unlinkable), (ii) closes the no-deniability clause, and (iii) achieves the binding by referencing a registered party at a single protocol step. Then  $\Pi$  performs a zero-knowledge proof of membership in the identity registry – over the recipient set at mint, or the issuer set at spend.*

*Proof.* By Lemma 15, (ii) forces  $\Pi$  to certify that  $eIss$  decrypts, under the addressed recipient’s registered identity material, to a registered *issuer* identity – binding to some key alone does not close the clause. This certificate references a registered party: the recipient (at mint), or the issuer

that *eIss* re-encrypts (at spend). By (iii) it is a single step’s reference, either in the clear or in zero knowledge. *In the clear is ruled out*: revealing the referenced identity or key on chain names a party and makes them linkable across mint and spend, violating (i); importing the spend-side ciphertext (or a deterministic commitment to it) into the mint to bind there re-links mint and spend, also violating (i). *Zero-knowledge is set-membership*: a reference certifying the referenced party is registered without revealing which is, by definition, a zero-knowledge proof of membership in the registry. As (i) excludes the in-clear case,  $\Pi$  performs this.  $\square$

## 9.4 The shipped system is the predicted branch

Proposition 16 permits exactly two homes for the membership proof, and the system’s own history visited both. An earlier design (retained in the retired drafting skeleton of this paper) placed it at *mint*: a per-leaf circuit opening the issuer’s credential, proving the recipient’s account-keyed credential a member of an account registry, and coupling the payload under the recipient’s *account key* – at roughly  $10^6$  R1CS per leaf, on top of the account-pinning brittleness of Sec. 5. The shipped system takes the *spend* branch, and the Identity-M inversion is what makes the branch practical:

- **The membership moves to where its prover is.** At spend the depositor is live, holds  $m_{\text{rec}}$ , and can prove membership of the relevant point – the *issuer’s* decrypted identity for A2 (closing the clause exactly as the proposition demands), the recipient’s own for A1. The proof is per-*spend*, not per-minted-leaf, and proving cost lands on the interested party’s own hardware; the chain only ever verifies.
- **The statement splits along its natural seam.** Account-binding and decryption-to-a-committed-point are sigma-protocol material (Theorem 9;  $\sim 90\text{K}$  gas to verify); only set-membership and the note tie need SNARKs at all. The composition is re-joined by deriving every public input from the shared objects – the discipline of Sec. 6.
- **Relocation creates one new obligation, and it is dischargeable.** Binding at spend means the verified ciphertext is the *depositor’s submission*, so it must be tied to the note being spent – otherwise a stolen opening redeems with a self-addressed ciphertext, and a coalition substitutes a nameable one for an un-nameable commitment. That obligation is Theorem 10’s two circuits, including the A1 face-pinning trick – the genuinely new constraint engineering this relocation demanded.

The result: the doubly-anonymous flavour, predicted expensive, costs a measured  $\sim 340\text{K}$  gas of extra verification per spend over the bearer flavour (Sec. 10) and seconds of off-chain proving – practical today. Whether restriction (iii) can be dropped – whether a binding amortised across steps, a two-party mint with a blinded recipient contribution, or a witness-encryption construction evades set-membership entirely – remains the paper’s central open problem (Sec. 11): an unconditional impossibility would prove the SNARK necessary; a loophole would be a cheaper design.

## 9.5 A mechanism remark: the recipient-burden posture

Before the tie circuits shipped, the design contemplated an economic backstop: an honest A2 recipient verifies the binding *at delivery* (decrypt *eIss*, check the registered identity) before accepting, so an audit regime with penalty  $L$  and probability  $q$  makes verify-then-accept dominant whenever  $qL$  exceeds the collusion benefit – and, by Corollary 13, the residual off-equilibrium outcome is only ever an un-nameable note held by an always-identifiable depositor, never a framed third party. With

the tie enforced on chain the mechanism is no longer load-bearing, but we record it: delivery-time verification remains correct wallet practice, and the analysis explains why even a deployment that lags the verifier rollout degrades to a bounded, recipient-borne risk rather than an open hole.

DRAFT

## 10 Implementation and Evaluation

Everything measured below is shipped, open, and reproducible from the repository: Solidity contracts (`Notes`, `IdentityRegistry`, the generated Groth16 verifiers behind thin adapters), circom circuits, the Python wallet reference, and an end-to-end Foundry suite (`NotesE2E`) that drives one full lifecycle per flavour with *every* verifier the real Groth16 artifact – real batch mint, real spend proof against the replayed tree, real deposit sigma, real bound membership, real note-binding tie.

### 10.1 What is shipped

| Component          | Realisation                                                                                                                         |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Identity substrate | <code>IdentityRegistry</code> : ElGamal credentials, PS verification, incremental Poseidon identity accumulator (commit-before-use) |
| Batch mint         | <code>mint_batch[_a2].circom</code> per pinned N; issuer Schnorr (public) / blinded re-encryption binding + leaf-tie (private)      |
| Spend              | <code>spend.circom</code> (flavour-agnostic, depth 20); nullifier Poseidon3( $\rho$ , <code>idHash</code> , 4242)                   |
| Deposit sigmas     | <code>verifyDepositCoupling</code> (A1/A2), <code>verifyDepositorBinding</code> (B1); EIP-196                                       |
| Bound membership   | <code>identity_membership_gltie.circom</code> ; public inputs derived from the sigma's P                                            |
| Note tie           | <code>note_binding.circom</code> (A2, 25 public inputs), <code>note_binding_a1.circom</code> (A1, 26: face at index 1)              |
| Spend entrypoints  | <code>spendCoupledA1</code> / <code>A2</code> / <code>B1</code>                                                                     |

### 10.2 Costs (measured)

Full-call gas for one mint (batch of one) and one coupled spend, per flavour, from the end-to-end suite (chain id 1, Cancun):

| Flavour                 | mint    | spendCoupled* |
|-------------------------|---------|---------------|
| A1 (addressed, public)  | 487,875 | 1,047,486     |
| A2 (addressed, private) | 607,639 | 1,041,100     |
| B1 (bearer, public)     | 487,875 | 701,474       |

Isolated per-phase verification gas:

| Verification phase                               | A1      | A2      | B1      |
|--------------------------------------------------|---------|---------|---------|
| note proof ( <code>spend.circom</code> , shared) | 227,256 | 227,256 | 227,256 |
| bound membership ( <code>gltie</code> )          | 253,120 | 253,120 | 253,120 |
| deposit sigma (EIP-196)                          | 89,866  | 89,866  | 128,734 |
| note tie ( <code>note_binding[_a1]</code> )      | 377,257 | 368,457 | –       |

The cost structure tells the paper's story in numbers. Both *addressed* flavours pay the tie – the price of "only  $M_{\text{rec}}$  can spend", about 340K over bearer – while issuer *anonymity* costs only the mint-side delta ( $\sim 120\text{K}$ , the blinded re-encryption binding): A2's spend is within half a percent of A1's. Set-membership, the provably necessary ingredient, is one constant 253K Groth16 verify regardless of registry size. Prover side: the tie circuits are 2.4M (A2) and 2.86M (A1) non-linear R1CS (fixed-base-only EC arithmetic; the A1 relation needs no witnessed point limbs at all since every point is a known multiple of  $G$ ), proving in seconds under rapidsnark on commodity Apple-silicon hardware, witness generation via the circom C++ calculator. All verification is constant-gas on chain; no participant ever waits on another's prover.

### 10.3 Verification methodology and artifact

The same vectors flow through three independent implementations – the Python wallet reference (py\_ecc-style BN254 arithmetic), the circom witness generators, and the Solidity verifiers – and the test suites assert byte-for-byte agreement: over 400 Foundry tests (including 18 on the tie verifiers alone and the 9-test all-real-verifier end-to-end lifecycle) and 330 wallet tests. The companion documents are executable: the walkthrough document runs every cryptographic step of all three

flavours in one seeded session, and this paper’s appendix witnesses re-run on each build. Honesty items: the Groth16 trusted setups currently use development entropy (a production Powers-of-Tau ceremony is a deployment prerequisite); formal verification of the circuits (beyond witness/constraint testing) is planned, not performed.

DRAFT

## 11 Discussion, Limitations, and Future Work

**Open problem (the necessity lower bound).** Proposition 16 is conditional on a single-binding-step restriction. The unconditional question – *must* any A2-private, collusion-resistant protocol perform registry set-membership ZK somewhere? – is open. The escape routes the restriction excludes are precisely the interesting future designs: bindings amortised across multiple steps; two-party mints where the recipient contributes a blinded verifiable-decryption witness without exposing the note ciphertext; witness-encryption-style constructions; and accountability that does not bind *eIss* at all (threshold or selective escrow – a strictly weaker trust model we reject for this system, since it reintroduces an opener).

### Limitations, stated plainly.

- The spend’s **face** and recipient account are public (an ERC-20 must pay someone a known amount); mitigations are operational – fixed denominations, relayed payouts – not cryptographic.
- Refusal-resilience and issuer-blindness are computational (DDH); archived ciphertexts are exposed to future cryptanalysis, and the KYC corpus is a harvest-now-decrypt-later target. A quantum-migration path for the identity substrate is future work with a deadline outside our control.
- The KYC issuer is trusted for *certification* (one person, one identity) and for honest-protocol issuance going forward (Sec. 4.3’s prospective caveat); Sybil identities from a corrupt certifier defeat naming, though never framing (Cor. 13).
- Trusted setup: development ptau until a ceremony is run.
- The registry accumulator is append-only in the current deployment; identity revocation/rotation semantics are designed but not shipped.

**The autonomy thesis, restated.** A jurisdiction adopting this design gets per-instrument compliance *stronger* than physical cash ever offered – every payment carries court-grade evidence, held by exactly the parties a lawful investigation would approach anyway – while forgoing the one thing every CBDC design quietly includes: the standing technical capability of population-scale financial surveillance. We contend the forgone capability is the point. Evidence that must be sought person by person, under process the subject can see and contest and – at personal cost – refuse, is the cryptographic shape of due process. A ledger that stays dark when its citizens withhold their keys is the cryptographic shape of the presumption that free people’s affairs are their own. The mathematics here does not decide which laws are just; it decides only that enforcing them runs through citizens, not over them – and that choice, once deployed, is not revocable by the preferences of any subsequent administration, because there is nothing left standing in the middle to compel.

## 12 Appendix A: Executable Theorem Witnesses

Each witness below executes, in this document's build, against the same `alberta_buck.wallet` reference implementation whose vectors drive the circom and Solidity test suites; the printed RESULTS blocks are regenerated on export (one seeded, deterministic session). They are sanity witnesses, not proofs: each exhibits the object whose existence (or impossibility-of-construction) the named result claims.

### 12.1 Setup: identities, accounts, registry

```
import random
from alberta_buck.wallet.bn254 import G1, ORDER, mul, add, neg, eq, rand_scalar
from alberta_buck.wallet.elgamal import elgamal_encrypt, elgamal_decrypt
from alberta_buck.wallet.unilateral_a2 import (
    IdentityTree, mint_unilateral_a2, make_receipt, verify_receipt,
)

_seed = random.Random(0xACC0)          # one deterministic transcript
rng = lambda: _seed.getrandbits(256)
CHAINID = 1

def account(m):
    # a registered account bound to identity m
    sk = rand_scalar(rng); pk = mul(G1, sk)
    return dict(m=m, M=mul(G1, m), sk=sk, pk=pk,
                E=elgamal_encrypt(mul(G1, m), pk, rand_scalar(rng)))

def pt(P):
    return f"({int(P[0]) % 10**8:>8d}..., {int(P[1]) % 10**8:>8d}...)"

def dec(R, C, sk):
    # ElGamal decrypt on raw points
    return add(C, neg(mul(R, sk)))

m_iss, m_rec = rand_scalar(rng), rand_scalar(rng)
issuer, rec = account(m_iss), account(m_rec)
M_rec = rec["M"]
tree = IdentityTree()
for a in (issuer, rec):
    tree.insert(a["M"])
print(f"issuer Identity    M_I    = {pt(issuer['M'])}")
print(f"recipient Identity M_rec = {pt(M_rec)}")
print(f"registry root       = {hex(tree.root())[20]}...")

issuer Identity    M_I    = (24518486..., 10011201...)
recipient Identity M_rec = (48294773..., 86086204...)
registry root      = 0x21b0a34077b3d8355c...
```

### 12.2 Witness: Lemma 15 (key-equality coupling is vacuous)

The attack object the lemma constructs: a note spendable by  $D$  whose issuer payload is keyed to a bogus key, satisfying the naive same-key constraint via the free witness.

```
sk_D, pk_D = rec["sk"], rec["pk"]
M_D        = rec["M"]
pk_bogus   = mul(G1, rand_scalar(rng))
r_n, r_p   = rand_scalar(rng), rand_scalar(rng)

# Free witness absorbs the constraint: M_rec' = M_D + r_n*(pk_D - pk_bogus)
M_rec_free = add(M_D, mul(add(pk_D, neg(pk_bogus)), r_n))
E_note = (mul(G1, r_n), add(M_rec_free, mul(pk_bogus, r_n))) # "same key" pk_bogus
eIss    = (mul(G1, r_p), add(issuer["M"], mul(pk_bogus, r_p))) # also under pk_bogus

assert eq(dec(*E_note, sk_D), M_D)          # note still decrypts for D: spendable
assert not eq(dec(*eIss, sk_D), issuer["M"]) # compelled sk_D cannot name the issuer
print("naive key-equality coupling: SATISFIED by the adversary, yet")
```

```

print(f" E_note decrypts under sk_D to M_D : True    (note spendable by D)")
print(f" eIss  decrypts under sk_D to M_I : False  (receipt destroyed)")
print("-> binding to 'a key' is vacuous; the binding must reach a REGISTERED identity")

```

```

naive key-equality coupling: SATISFIED by the adversary, yet
  E_note decrypts under sk_D to M_D : True    (note spendable by D)
  eIss  decrypts under sk_D to M_I : False  (receipt destroyed)
-> binding to 'a key' is vacuous; the binding must reach a REGISTERED identity

```

## 12.3 Witness: Theorems 11 / 12 (A2 receipt; un-nameable is un-spendable)

The honest A2 path: the recipient alone produces a receipt naming both parties. The colluding path: an *eIss* keyed to a throwaway point decrypts to garbage that is *not* a registry member – the membership gate has no witness, so the note cannot be spent.

```

note = mint_unilateral_a2(issuer["sk"], issuer["E"], M_rec,
                          v=1000, rho=rand_scalar(rng),
                          issuer=0xA11CE, chainid=CHAINID, rng=rng)
receipt = make_receipt(m_rec, note, issuer=0xA11CE, chainid=CHAINID,
                      tree=tree, rng=rng)
res = verify_receipt(receipt, issuer["pk"], issuer["E"], tree.root(), tree)
assert res.valid and eq(res.issuer_M, issuer["M"]) and eq(res.recipient_M, M_rec)
print(f"A2 receipt: {res.reason} -- names issuer {pt(res.issuer_M)}")
print(f"          and recipient {pt(res.recipient_M)}, value {res.value}")

# Collusion: key eIss to a throwaway point.
pk_throw = mul(G1, rand_scalar(rng))
eIss_bad = elgamal_encrypt(issuer["M"], pk_throw, rand_scalar(rng))
M_garbage = elgamal_decrypt(eIss_bad, m_rec)
print(f"colluding eIss decrypts under m_rec to {pt(M_garbage)}")
print(f" registered identity? {tree.contains(M_garbage)} -> membership unprovable;"
      f" deposit gate reverts: un-nameable => un-spendable")

A2 receipt: VALID -- names issuer (24518486..., 10011201...)
                and recipient (48294773..., 86086204...), value 1000
colluding eIss decrypts under m_rec to (87647643..., 90375864...)
registered identity? False -> membership unprovable; deposit gate reverts: un-nameable => un-spendable

```

## 12.4 Witness: Theorem 10 (A2 layout – substitution cannot bind)

The colluders *can* build a tie witness over their bogus ciphertext (they know its randomness) – but its *idHash* is not the spent note's, so the in-circuit nullifier disagrees with the one the spend consumed, and the on-chain public-input derivation rejects the proof.

```

from alberta_buck.wallet.note_binding import make_note_binding_witness

s, b = rand_scalar(rng), rand_scalar(rng)
w_honest = make_note_binding_witness(
    rho=note.opening.rho, eNote=note.eNote, eIssCommitted=note.eIss,
    s=s, m_rec=m_rec, b=b, M_I=note.M_I, r_iss=note.r_prime)

r_bad = rand_scalar(rng)
eIss_sub = elgamal_encrypt(issuer["M"], M_rec, r_bad) # nameable substitute
w_sub = make_note_binding_witness(
    rho=note.opening.rho, eNote=note.eNote, eIssCommitted=eIss_sub,
    s=rand_scalar(rng), m_rec=m_rec, b=b, M_I=issuer["M"], r_iss=r_bad)

print(f"spent note's nullifier      = {hex(int(w_honest['nullifier']))[:20]}...")
print(f"substituted-tie nullifier    = {hex(int(w_sub['nullifier']))[:20]}...")
assert w_sub["nullifier"] != w_honest["nullifier"]
print("-> public-input mismatch: a substituted ciphertext cannot bind THIS spend")

```

```

spent note's nullifier      = 0x1e63df3688a6e7f638...
substituted-tie nullifier   = 0x21cbb7dfd6effd1be5...
-> public-input mismatch: a substituted ciphertext cannot bind THIS spend

```

## 12.5 Witness: Theorem 10 (A1 layout – the public face pins the identity)

The production A1 witness builder asserts the full relation: it builds for the addressed identity, and *refuses* for any other – even for a thief holding the complete opening including the note randomness – because the public face leaves no free plaintext to absorb the re-keying.

```

from alberta_buck.wallet.unilateral_a1 import mint_unilateral_a1
from alberta_buck.wallet.note_binding import make_note_binding_a1_witness

k = rand_scalar(rng)
sigma_R, sigma_s = mul(G1, k), (k + rand_scalar(rng) * rand_scalar(rng)) % ORDER
note_a1 = mint_unilateral_a1(M_rec, v=100, rho=rand_scalar(rng),
                             m_issuer=m_iss, sigma_R=sigma_R, sigma_s=sigma_s,
                             rng=rng)
t, b1 = rand_scalar(rng), rand_scalar(rng)
w_a1 = make_note_binding_a1_witness(
    rho=note_a1.opening.rho, eNote=note_a1.eNote, v=note_a1.opening.v,
    m_issuer=m_iss, sigma_R=sigma_R, sigma_s=sigma_s,
    r_note=note_a1.r_note, m_rec=m_rec, t=t, b=b1)
print(f"A1 tie witness for the ADDRESSED identity: built "
      f"(public face v = {w_a1['v']})")

m_thief = rand_scalar(rng)      # thief holds the WHOLE opening, incl. r_note
try:
    make_note_binding_a1_witness(
        rho=note_a1.opening.rho, eNote=note_a1.eNote, v=note_a1.opening.v,
        m_issuer=m_iss, sigma_R=sigma_R, sigma_s=sigma_s,
        r_note=note_a1.r_note, m_rec=m_thief, t=t, b=b1)
    print("thief witness: BUILT (must not happen!)")
except AssertionError as exc:
    print(f"A1 tie witness for a thief identity: REFUSED ({exc})")

A1 tie witness for the ADDRESSED identity: built (public face v = 100)
A1 tie witness for a thief identity: REFUSED (eNote.C != v*G + rn*M_rec)

```

## 12.6 Reproducing the full system

The on-chain halves of every claim are exercised by the repository's suites; the end-to-end lifecycle (real batch mint, real spend, real sigma, real bound membership, real tie, per flavour) reproduces with:

```

$ make nix-snark-note-binding nix-snark-note-binding-a1  # tie circuits + setups
$ make nix-snark-e2e-fixtures                             # all-real-verifier fixtures
$ nix develop --command forge test --match-contract NotesE2E -vv
[PASS] ... 9 tests: lifecycle, double-spend revert, per-phase gas, x3 flavours

```